HLD-as-Source-of-Intent: Bootstrapping Intent-Based Networks from High-Level Design

Marc Bruyère-Yamada
Télécom SudParis
JFLI
France / Japan
email TBD

Co-authors TBD
Affiliations TBD
email TBD

***Abstract*—Intent-based abstractions are increasingly central to network automation, but they are usually introduced only after a network has already been designed or deployed. This paper asks whether a High-Level Design (HLD) document can instead serve as the earliest explicit intent artefact for bootstrapping a network from design to its first operational state. Existing work on intent-based networking, configuration generation, and LLM-assisted network management leaves this design-time step only partially addressed because it typically starts from running infrastructures, lower-level specifications, or underconstrained natural-language requests. We propose an *Intent Zero* workflow in which a structured HLD is translated through intermediate design objects into vendor-aware implementation artefacts, validated in a lab or digital-twin environment, and approved before deployment. We instantiate this workflow as a prototype closed loop for SME networks that combines agentic orchestration, structured source-of-truth management, memory-backed coordination, and human checkpoints, and we use it to define an evaluation methodology across multiple HLDs, vendor contexts, and model settings. This perspective reframes the design-to-deployment gap as an intent-based management problem and provides a practical foundation for linking early network realization with later operational assurance loops.**

***Index Terms*—intent-based networking, high-level design, network automation, closed-loop management, network design, digital twin, agentic systems**

I. Introduction

Modern network and service management is being pushed toward higher levels of automation, programmability, and intelligence by the growing scale, heterogeneity, and service criticality of contemporary infrastructures [1]. In enterprise environments, this pressure is visible well before day-to-day operations begin: new sites, segmentation policies, service zones, resilience targets, and security constraints must first be designed, documented, and turned into a deployable network. Yet this early phase is still largely driven by human-authored High-Level Design (HLD) documents and manual engineering practices, despite long-standing evidence that enterprise network design remains complex, customized, and insufficiently systematized [2], [3]. The resulting gap sits at the intersection of configuration management, policy- and intent-based management, orchestration, software engineering methodologies, and digital twins for networks and services.

The specific problem considered in this paper is the transition from HLD to first network realization. Existing IBN work most often assumes that a network already exists and studies how high-level intents are translated into operational actions such as fulfillment, assurance, and remediation [4]. By contrast, real enterprise projects usually begin from an HLD that captures architectural intent but is not itself executable. Classical enterprise design work has treated planning, dimensioning, and reachability design as structured problems [2], [3], while recent intent and LLM-based systems have focused on intent translation, configuration updating, or workflow assistance once a target environment, a request, or an existing configuration is already available [5]–[7]. The question we address is therefore different: can a structured HLD serve as the earliest machine-actionable source of intent for bootstrapping a small or medium enterprise network from design to first validated deployment?

In this paper, we show that the answer can be yes, provided that the HLD is treated as a structured intent artefact rather than as static documentation. First, we reframe IBN from its usual operational focus toward a design-to-deployment setting and position the HLD as the earliest explicit intent artefact in the network lifecycle, consistent with the conceptual foundations of RFC 9315 and the classification perspective of RFC 9316 [4], [8]. Second, we propose a high-level agentic architecture for HLD-driven network realization in which specialized agents collaborate through a shared workflow and use short-term and long-term memory support, inspired by recent advances in multi-agent LLM systems for network management [9], [10]. Third, we instantiate this proposition as a prototype closed loop that transforms HLD information into normalized design objects, implementation artefacts, deployment actions, and as-built feedback. Fourth, we define an evaluation methodology based on multiple SME HLDs, heterogeneous vendor profiles, human-in-the-loop checkpoints, and, when feasible, sensitivity to model families. Taken together, these contributions position HLD-driven bootstrap as a concrete and testable extension of intent-based management toward what we term the *T0*, or *Intent Zero*, phase of the network lifecycle.

Our approach differs from previous work in four ways. Compared with operational IBN frameworks and intent handlers, we begin from a design document rather than from a deployed network, a service request, or a controller-facing

intent object [4], [5]. Compared with LLM-based configuration studies, we are not asking only whether a model can synthesize commands or update configurations from prompts; we focus on the larger design-to-deployment pipeline, where ambiguity, validation, traceability, and lifecycle structure matter as much as generation quality [6], [7], [11]. Compared with hyperscale design materialization systems, our target is enterprise HLD-driven bootstrap rather than data-center-specific realization [12], [13]. Finally, compared with production multi-agent assistants, the contribution here is not another network copilot, but the positioning of HLD-as-source-of-intent for initial network realization [9].

The rest of the paper moves from foundations to realization and evaluation. Section II sets the lifecycle background, revisits IBN concepts relevant to this work, and positions the paper against related literature. Section III states the proposition at a high level and defines the scope of the closed loop. Section IV details the input model, transformation workflow, and implementation choices. Section V presents the experimental questions, scenarios, and metrics. Section VI discusses limitations, implications, and coexistence with later operational IBN loops, before Section VII concludes.

II. Background and Motivation

A. Network Lifecycle and the Role of HLD

Enterprise network realization usually follows a broad lifecycle that starts from business needs and requirements, proceeds through architecture, HLD, lower-level design and implementation planning, and only then reaches deployment, validation, and operations. Within this lifecycle, the HLD is often the first artefact that consolidates site structure, service zones, segmentation goals, interconnection principles, resilience expectations, and acceptance criteria into a coherent description of the target network. In practice, however, the HLD remains authoritative for humans but inert for machines. Its content must still be reinterpreted, decomposed, and re-entered through manual engineering steps before any deployable artefact exists, which makes the transition from design to implementation costly, slow, and error-prone, especially in multi-vendor environments [2], [3], [14].

B. IBN Concepts Relevant to This Paper

RFC 9315 defines IBN as a closed-loop system that converts high-level intent into network behavior and continuously checks conformance through fulfillment and assurance functions [4]. It also distinguishes an inner autonomous loop from an outer human loop, a decomposition that remains useful here even though this paper focuses on the earliest fulfillment stages rather than on steady-state operations. RFC 9316 complements this view by classifying intents according to stakeholder perspective, abstraction level, and lifecycle role [8]. For this paper, the key question is therefore not only what intent is, but also when intent becomes operationally actionable. Our claim is that the HLD can be treated as such an artefact earlier than is usually assumed, provided that it is structured enough to support translation, validation, and traceability.

C. Related Work and Research Gap

Most IBN literature and platforms assume that a network already exists and concentrate on policy translation, service steering, orchestration, or closed-loop assurance over a running infrastructure [4], [5]. In parallel, recent LLM-assisted work has studied intent translation, configuration updating, workflow assistance, and model benchmarking, showing both the promise and the fragility of generation-centric approaches [6], [7], [9], [11]. A complementary line of work on multi-level abstraction and design materialization, from MALT to recent hyperscale realizers, shows the value of explicit intermediate representations between design and deployment [12], [13]. What remains largely unaddressed is the specific white-field problem considered here: treating an enterprise HLD itself as the source of intent for initial network realization. In that sense, the present work is closer to design-time IBN, or *Intent Zero*, than to classical operational IBN.

III. Proposition: Overview

A. Core Proposition

The core proposition of this paper is that a structured HLD can serve as the first machine-actionable intent artefact for enterprise network realization. The input to the workflow is therefore not an already deployed infrastructure, nor a controller-native intent object, but an HLD that describes the target SME network and, when needed, its technology constraints. The expected output is a first validated network realization, including topology artefacts, vendor-aware configurations, a deployment plan, and as-built feedback. The central hypothesis is that, if the HLD is sufficiently structured, it can act as the earliest formalized intent artefact in the lifecycle without requiring operators to re-enter the same design decisions into a separate controller-specific model. The scope remains deliberately bounded: the goal is not to solve the entire operational lifecycle, but to address the transition from design intent to first deployable and validated network state.

B. Closed-Loop View

At a high level, the proposed loop starts by ingesting and parsing the HLD into structured design objects, which are then normalized into an internal representation of the intended network. From that representation, the workflow derives policy, topology, addressing, segmentation, and implementation artefacts that can be rendered into vendor-specific outputs. Before any production push, the result is validated in a lab or digital-twin environment so that mismatches can be detected while the network is still in a design-to-deployment phase rather than in live operation. The loop then produces deployment and validation outputs for human review whenever changes are high impact, and finally records the observed result in an as-built view that can serve as a cleaner handoff toward later operations. In that sense, the loop already exhibits the core IBN pattern of intent transformation, realization, and feedback, but it does so at the earliest stage of the lifecycle.

C. Why a Closed Loop at T0?

The motivation for closing the loop already at *T0*, or *Intent Zero*, is straightforward. Design artefacts and implementation artefacts can drift long before a network reaches normal operation, and many of the most expensive mismatches originate precisely in that early translation phase. A design-time loop can expose missing constraints, unsupported vendor features, invalid topology assumptions, and inconsistent security intent before they become production incidents. A lab or digital-twin validation stage further reduces the risk of turning an HLD directly into live changes by inserting an explicit realization-and-check step between design and deployment. More broadly, treating initial realization as a closed loop makes the eventual handoff to operational IBN cleaner: the network reaches its first deployed state with better traceability, better validation evidence, and a more explicit relation between intended and realized behavior.

IV. Details

A. Input Model: HLD or Structured Input File

Our current working assumption is that the entry artefact remains an HLD, but an HLD that is explicitly structured for reproducibility and parsing rather than written as unconstrained prose. At minimum, such a document should expose stable sections for scope, sites, zones, topology intent, addressing, segmentation, external connectivity, security rules, critical services, platform constraints, and validation expectations. One open design question is whether all technology constraints should remain inside the HLD or whether they should be externalized into a companion constraint file. A closely related question concerns formalization: Markdown with tables is a practical candidate because it remains easy for network architects to author, but alternative notations such as PlantUML, Mermaid, or other machine-readable design formats may ultimately offer stronger guarantees for parsing and transformation. For the purposes of this paper, the essential point is not to commit prematurely to one notation, but to require that the input artefact be structured enough to support deterministic extraction and downstream validation.

B. Transformation Workflow

The workflow begins with design parsing, whose role is to extract structured entities and relations from the HLD instead of treating the document as free-form text. These extracted elements are then normalized into an internal representation that separates invariant design choices from platform-dependent constraints. On that basis, the system derives concrete network intents such as segmentation, reachability, redundancy, and service exposure, which in turn drive implementation synthesis. Implementation synthesis produces vendor-aware artefacts, low-level configurations, topology descriptors, and deployment plans suitable for lab realization. The resulting artefacts are then validated in an executable environment, where connectivity, segmentation, and consistency checks can be performed and discrepancies recorded. Finally, the realized state is fed back into an as-built view, so that the end product of the workflow is not just a set of generated configurations but a traceable relation between the original HLD, the derived implementation, and the observed outcome.

C. Prototype Realization Choices

The current prototype realizes this workflow through an agentic orchestration harness that decomposes the pipeline into specialized tasks. This decomposition is useful because the overall problem naturally combines parsing, normalization, derivation, rendering, validation, and feedback, each of which benefits from explicit interfaces and traceable intermediate outputs. At the same time, the present paper intentionally treats the multi-agent structure as a means of realization rather than as the main claim. A graph-backed source of truth is used to preserve cross-layer relations between design objects, implementation artefacts, deployment state, and validation results, which is consistent with prior work on multi-level representations and structured intent handling [5], [12]. Additional memory layers can support rationale capture, traceability, and human interaction, but they remain secondary to the main contribution unless they can be shown to materially affect the design-to-deployment loop itself. Finally, vendor-specific rendering and deployment backends are necessary to demonstrate that the approach is not tied to a single target platform and can support heterogeneous realization paths [6], [13], [14].

D. Human-in-the-Loop and Governance

The workflow is intended to remain largely automated for low-risk transformations, but this does not remove the need for explicit governance. A human checkpoint remains necessary whenever the transformation has a high operational or architectural impact, for instance when it introduces new devices, changes topology structure, creates security-sensitive exposure, or leaves unresolved implementation ambiguity. This point is important methodologically because one dimension of the evaluation is not only whether the workflow produces technically correct artefacts, but also how much and what kind of human intervention is still required. In practice, this is also where the hardest real-world constraints tend to surface: closed-vendor documentation, incomplete device models, and ambiguous HLD wording often reveal themselves not as parsing failures, but as governance decisions about whether the system can proceed autonomously or must escalate.

V. Experimental Evaluation

A. Objectives and Research Questions

The objective of the evaluation is to determine whether HLD-driven bootstrap is viable as a repeatable network realization workflow rather than as a single proof-of-concept demonstration. Four research questions structure this evaluation. First, can a structured HLD provide enough information to generate a first deployable SME network without manual re-entry into another intent model? Second, how robust is the workflow across multiple HLDs, topology shapes, and vendor or technology constraints? Third, where is human intervention

still required, and is that intervention primarily caused by HLD ambiguity, missing constraints, or implementation limitations? Fourth, to what extent are the results dependent on a specific model family, toolchain, or vendor backend? Together, these questions aim to assess not only end-to-end feasibility, but also the practical boundaries of the approach.

B. Experimental Setup

The target domain of the experiments is small and medium enterprise networks, which provide enough heterogeneity to make the problem meaningful while remaining tractable for repeated evaluation. Validation is performed in Containerlab or in another comparable digital-twin infrastructure so that the full workflow can be exercised without requiring a production deployment path. To assess cross-platform realizability, the setup should include multiple vendor profiles, for example software routers, Linux-based network functions, and model-driven platforms. A central requirement of the setup is reproducibility: the same HLD input must lead to the same expected artefacts and validation checks under a controlled execution environment, making it possible to distinguish true workflow limitations from incidental execution noise.

C. Scenario Corpus

The scenario corpus is intended to contain roughly ten HLDs of increasing complexity, spanning cases such as single-site and multi-site designs, DMZ exposure, guest access, service segmentation, redundancy, and vendor heterogeneity. The corpus should combine synthetic scenarios with industrially inspired HLDs whenever such material can be obtained in a reusable form. In addition to nominal cases, the evaluation should include stress cases that expose the true limits of the workflow, such as underspecified HLDs, conflicting constraints, unsupported vendor features, or dependencies on closed-vendor documentation. This mix is important because a convincing evaluation must show not only where the pipeline succeeds, but also the recurring conditions under which it becomes uncertain or requires escalation.

D. Metrics

The primary metrics are the end-to-end success rate per HLD, the time to first validated network realization, and the coverage of HLD elements that are successfully translated into deployment artefacts. These metrics should be complemented by validation pass rates over connectivity, segmentation, reachability, and service tests, since a generated artefact is useful only if it satisfies the intended network behavior. A second family of metrics concerns workflow friction: the number of manual interventions per scenario, the nature of those interventions, and their underlying causes. Reproducibility across repeated runs is also essential, especially when optional LLM-assisted components are enabled. Where relevant, additional cost-oriented indicators such as token consumption, latency, and sensitivity to model family can help quantify the operational trade-offs introduced by agentic or generative components.

E. Comparative and Ablation Angles

Several comparative and ablation angles can strengthen the evaluation. A first comparison is between structured HLD input and less structured variants, in order to quantify the value of formalization rather than merely assume it. A second comparison concerns input organization: one combined artefact versus a split HLD-plus-constraints scheme. A third comparison concerns realization context, contrasting validation-only digital-twin execution with a path that targets a more realistic deployment backend. Finally, optional components such as memory support, orchestration strategy, or model family can be ablated to determine which parts of the system are essential to the core claim and which parts mainly improve usability, traceability, or engineering convenience.

VI. Discussion

A. Positioning

The proposal developed in this paper should not be read as a replacement for operational IBN. Instead, it introduces a preceding phase of design-time intent realization, or *Intent Zero*, whose role is to prepare a network before classical assurance and operations begin. This distinction resolves an apparent tension in the overall framing: the system remains grounded in IBN concepts and RFC terminology, but it is applied earlier in the lifecycle than most existing IBN work. In that sense, the contribution is to extend the intent-based view of network management toward the point where networks are still being made, not only managed.

B. Limits and Threats to Validity

Several limitations should be stated explicitly. First, the results are likely to depend strongly on the structure and quality of the HLD itself: if the design artefact is incomplete, inconsistent, or too informal, the workflow may fail for reasons that are orthogonal to the realization architecture. Second, a lab or digital twin remains only a proxy for production infrastructure, even though it is highly valuable for pre-deployment validation. Third, vendor closedness, missing YANG or API models, and incomplete documentation may still force extra human intervention and reduce the degree of autonomy that can be achieved in practice. Finally, the chosen orchestration architecture may affect engineering efficiency, observability, and ease of extension without changing the central claim that a structured HLD can anchor the design-to-deployment loop.

C. Implications

The implications of this work are twofold. For IBN research, the results suggest that intent should be studied not only as an operational abstraction but also as a design-time artefact whose role begins before the network exists. For practitioners, they suggest that a disciplined HLD format may reduce the cost of moving from architecture to implementation by making more of the early lifecycle machine-actionable. More broadly, the framework outlined here creates a natural bridge toward future extensions: once an initial network has been realized with explicit traceability between intended and deployed states,

the same foundation can be extended toward an operational IBN loop, continuous assurance, and post-deployment design evolution.

VII. Conclusion

This paper argues that the HLD can be treated as the earliest explicit source of network intent and that the gap between design and deployment can therefore be framed as an intent-based management problem rather than only as a documentation or automation problem. From that perspective, we outlined a closed-loop workflow for SME environments that transforms a structured HLD into normalized design objects, implementation artefacts, validation evidence, and an initial as-built network state.

The next steps are to finalize the HLD formalization strategy, complete the experimental campaign across multiple HLDs and vendor contexts, and refine the articulation between *Intent Zero* and subsequent operational IBN loops. More broadly, the objective is to make network design more explicit, traceable, and machine-actionable from the very first realization of the network.

References

[1] A. Clemm, M. F. Zhani, and R. Boutaba, "Network management 2030: Operations and control of network 2030 services," *J. Netw. Syst. Manage.*, vol. 28, no. 4, p. 721–750, Oct. 2020. [Online]. Available: https://doi.org/10.1007/s10922-020-09517-0

[2] E. Drakopoulos, "Enterprise network planning and design: methodology and application," *Computer Communications*, vol. 22, no. 4, pp. 340–352, 1999. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0140366498001583

[3] Y.-W. E. Sung, S. G. Rao, G. G. Xie, and D. A. Maltz, "Towards systematic design of enterprise networks," in *Proceedings of the 2008 ACM CoNEXT Conference*, ser. CoNEXT '08. New York, NY, USA: Association for Computing Machinery, 2008. [Online]. Available: https://doi.org/10.1145/1544012.1544034

[4] A. Clemm, L. Ciavaglia, L. Z. Granville, and J. Tantsura, "Intent-Based Networking - Concepts and Definitions," RFC 9315, Oct. 2022. [Online]. Available: https://www.rfc-editor.org/info/rfc9315

[5] P. Alcock, R. Anand, C. Rotsos, and N. Race, "Swift: Semantic web intent framework for intent translation," in *NOMS 2025-2025 IEEE Network Operations and Management Symposium*, 2025, pp. 01–09.

[6] R. Han, J. Wang, H. Sun, Z. Jiang, Q. Qi, Z. Zhuang, Y. Zhang, and J. Liao, "Network copilot: Intent-driven network configuration updating for service guarantee," in *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, 2025, pp. 1–10.

[7] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, "Netconfeval: Can llms facilitate network configuration?" *Proc. ACM Netw.*, vol. 2, no. CoNEXT2, Jun. 2024. [Online]. Available: https://doi.org/10.1145/3656296

[8] C. Li, O. Havel, A. Olariu, P. Martinez-Julia, J. C. Nobre, and D. Lopez, "Intent Classification," RFC 9316, Oct. 2022. [Online]. Available: https://www.rfc-editor.org/info/rfc9316

[9] Z. Wang, S. Lin, G. Yan, S. Ghorbani, M. Yu, J. Zhou, N. Hu, L. Baruah, S. Peters, S. Kamath, J. Yang, and Y. Zhang, "Intent-driven network management with multi-agent llms: The confucius framework," in *Proceedings of the ACM SIGCOMM 2025 Conference*, ser. SIGCOMM '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 347–362. [Online]. Available: https://doi.org/10.1145/3718958.3750537

[10] K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O'Sullivan, and H. D. Nguyen, "Multi-agent collaboration mechanisms: A survey of llms," 2025. [Online]. Available: https://arxiv.org/abs/2501.06322

[11] R. Mondal, N. Bjorner, T. Millstein, A. Tang, and G. Varghese, "Tackling ambiguity in user intent for llm-based network configuration synthesis," in *Proceedings of the 24th ACM Workshop on Hot Topics in Networks*, ser. HotNets '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 176–183. [Online]. Available: https://doi.org/10.1145/3772356.3772402

[12] J. C. Mogul, D. Goricanec, M. Pool, A. Shaikh, D. Turk, B. Koley, and X. Zhao, "Experiences with modeling network topologies at multiple levels of abstraction," in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. Santa Clara, CA: USENIX Association, Feb. 2020, pp. 403–418. [Online]. Available: https://www.usenix.org/conference/nsdi20/presentation/mogul

[13] Y. Cai, J. Li, K. Akpinar, T. Li, H. Morsy, J. Wilson, S. Khaunte, Y. Xia, and Y. Zhang, "Matryoshka: Realizing hyperscale data center network design for the AI era," in *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. Renton, WA: USENIX Association, May 2026, pp. 2095–2110. [Online]. Available: https://www.usenix.org/conference/nsdi26/presentation/cai

[14] W. Enck, P. McDaniel, S. Sen, P. Sebos, S. Spoerel, A. Greenberg, S. Rao, and W. Aiello, "Configuration management at massive scale: System design and experience," in *2007 USENIX Annual Technical Conference (USENIX ATC 07)*. Santa Clara, CA: USENIX Association, Jun. 2007. [Online]. Available: https://www.usenix.org/conference/2007-usenix-annual-technical-conference/configuration-management-massive-scale-system